

# Windows 桌面安装包构建与自动更新完整指南

Next.js 纯前端 + Python FastAPI 伴侣进程 · 工业级发布与热更新全方案

|                                                                                                          |                                                                                                                      |                                                                                                                |
|----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
|  项目名称: XC_OmniBox (万象箱) |  核心架构: Electron + Next.js + FastAPI |  分发交付: Windows NSIS 静默更新安装包 |
|----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|

把当前项目（Next.js 前端 + Python FastAPI 后端）改造成带有自动更新功能的 **Windows 桌面安装包**，分为以下 **5 个完整阶段**。下面为你梳理从准备环境、日常热更新开发、打包安装包、到最后发布自动更新的**端到端全流程**：

## 阶段一：项目环境与依赖准备

在当前项目根目录（Tools/）下，安装桌面端外壳的核心依赖模块：

```
●●● BASH — 终端命令：安装 Electron 与桌面端外壳依赖

# 安装 electron 开发核心、打包构建器、以及支持自动更新的模块
npm install --save-dev electron electron-builder concurrently wait-on
npm install electron-updater tree-kill
```

安装的核心依赖库及其在架构中的关键职责说明如下：

- **electron-builder**：负责把所有前端静态文件、主进程代码与 Python 独立运行时压缩封装成 Windows NSIS 一键安装程序（.exe）。
- **electron-updater**：负责在客户端后台静默检查版本差分、下载更新包、SHA512 哈希校验并在用户重启时自动替换升级。
- **tree-kill**：确保用户关闭软件或崩溃退出时，能够沿进程树递归彻底杀死后台常驻的 Python FastAPI 进程，杜绝后台僵尸进程与端口占用。

 **图标资源准备规范 (resources/icon.ico)**

准备一张 **256×256 像素** 的透明背景高清 Logo，使用转换工具生成标准的 **icon.ico** 多分辨率图标文件，保存在项目目录 **resources/icon.ico**。打包工具将自动注入至 Windows 快捷方式及应用程序图标中。

## 阶段二：Python 后端独立化 (PyInstaller)

目标终端用户的电脑上绝大多数没有安装 Python 环境。因此，必须将后端的 FastAPI 框架、Uvicorn 服务以及 PyMuPDF 等算法依赖，编译打包成一个**完全独立的免安装目录**，随客户端一并分发：

### 1. 在 backend/ 虚拟环境中安装 PyInstaller

```
●●● BASH - 终端命令：在后端环境中安装 PyInstaller
pip install pyinstaller
```

### 2. 执行 Python 后端打包命令

编写一个打包命令/脚本，将 `app/main.py` 打包为免安装目录：

```
●●● BASH - 终端命令：PyInstaller 目录模式打包
pyinstaller --noconfirm --onedir --name "omni-backend" --add-data "app;app" app/main.py
```

#### 💡 打包产物与分发路径说明

打包完成后，会在 `backend/dist/omni-backend/` 目录下生成包含独立 Python 运行时及 `omni-backend.exe` 的免安装文件夹。

这个文件夹后续会自动被 Electron 的 `extraResources` 机制一起打进最终安装包，安装后内置于资源目录，无需终端用户手动安装 Python。

#### ⚙️ PyInstaller 关键参数解析

- `--noconfirm`：静默覆盖已有构建输出，防止因交互确认提示导致自动化打包脚本阻塞挂起。
- `--onedir`：生成免安装目录格式。相比单文件 (`--onefile`) 启动耗时大幅缩减 90% 以上（无需每次运行向临时目录解压数百兆运行时）。
- `--add-data "app;app"`：将 FastAPI 应用源码目录及内部静态依赖完整打包进独立执行目录。
- `--name "omni-backend"`：指定输出可执行文件名，与后文 Electron 主进程中的子进程拉起逻辑精准匹配。

## 阶段三：编写 Electron 主进程（支持开发热更 & 生产静默运行）

在项目根目录下新建 **electron/** 目录，编写桌面端外壳生命周期与进程控制逻辑：

### 1. electron/main.js（核心主进程控制器）

```

●●● JS - electron/main.js

const { app, BrowserWindow, ipcMain } = require('electron');
const path = require('path');
const { spawn } = require('child_process');
const treeKill = require('tree-kill');

const isDev = process.env.NODE_ENV === 'development';
let mainWindow = null;
let backendProcess = null;
const BACKEND_PORT = 18520; // 专属本地端口，防止和 8000 冲突

// 1. 启动 Python 伴侣进程
function startBackend() {
  const backendExe = isDev
    ? path.join(__dirname, '../backend/dist/omni-backend/omni-backend.exe')
    : path.join(process.resourcesPath, 'backend', 'omni-backend.exe');

  backendProcess = spawn(backendExe, ['--port', String(BACKEND_PORT)]);
}

// 2. 创建主窗口
function createWindow() {
  mainWindow = new BrowserWindow({
    width: 1280,
    height: 850,
    icon: path.join(__dirname, '../resources/icon.ico'),
    webPreferences: {
      preload: path.join(__dirname, 'preload.js'),
      nodeIntegration: false,
      contextIsolation: true,
    },
  });
}

if (isDev) {
  // 【开发阶段】：直接连 Next.js 开发服务器，支持毫秒级热更新！
  mainWindow.loadURL('http://localhost:3000');
  mainWindow.webContents.openDevTools(); // 自动打开 F12 控制台
} else {
  // 【生产发布】：加载打包好的本地 Next.js 静态文件
  mainWindow.loadFile(path.join(__dirname, '../frontend/out/index.html'));
}

app.whenReady().then(() => {
  startBackend();
  createWindow();
});

// 3. 彻底退出处理：关闭窗口时杀死后台 Python
app.on('before-quit', () => {
  if (backendProcess && backendProcess.pid) {
    treeKill(backendProcess.pid, 'SIGKILL');
  }
});

```

## 2. electron/preload.js (安全通讯通道)

```

● ● ● JS - electron/preload.js

const { contextBridge, ipcRenderer } = require('electron');

contextBridge.exposeInMainWorld('electronAPI', {
  checkForUpdates: () => ipcRenderer.invoke('check-for-updates'),
  startDownload: () => ipcRenderer.invoke('start-download'),
  quitAndInstall: () => ipcRenderer.invoke('quit-and-install'),
  onUpdateStatus: (callback) => ipcRenderer.on('update-status', (e, val) => callback(val)),
});

```

## 阶段四：前端适配与自动更新交互

### 1. Next.js 开启纯静态导出

修改 `frontend/next.config.mjs`，加上静态导出设置：

```

● ● ● JS - frontend/next.config.mjs

/** @type {import('next').NextConfig} */
const nextConfig = {
  output: 'export', // 导出静态 HTML/JS
  images: { unoptimized: true },
};

export default nextConfig;

```

### 2. 前端请求地址兼容

前端请求 Python API 时，判断如果是桌面环境，自动请求 `http://127.0.0.1:18520`：

```

● ● ● TS - API 地址自适应逻辑

export const API_BASE_URL = typeof window !== 'undefined' && (window as any).electronAPI
  ? 'http://127.0.0.1:18520'
  : 'http://127.0.0.1:8000';

```

### 前端与桌面端通信架构设计要点

- **上下文隔离保障**：关闭 `nodeIntegration` 并启用 `contextIsolation`，仅通过 `preload.js` 向渲染进程暴露受控 API，杜绝 XSS 漏洞直接执行系统命令的安全隐患。
- **纯静态导出架构**：Next.js 输出纯粹的 HTML/JS/CSS，无需在客户端运行 Node.js 服务端渲染，极大降低内存占用并实现秒开体验。

### 3. 主进程接入 electron-updater (监听更新事件)

在 `electron/updater.js` 中配置自动更新生命周期事件与 IPC 处理句柄：

```
JS - electron/updater.js

const { autoUpdater } = require('electron-updater');
const { ipcMain } = require('electron');

function setupAutoUpdater(win) {
  autoUpdater.autoDownload = false; // 用户点击后再下载

  // 监听到新版本
  autoUpdater.on('update-available', (info) => {
    win.webContents.send('update-status', { step: 'available', version: info.version });
  });

  // 下载进度 (0% ~ 100%)
  autoUpdater.on('download-progress', (progress) => {
    win.webContents.send('update-status', { step: 'downloading', percent: progress.percent });
  });

  // 下载完毕, 准备安装
  autoUpdater.on('update-downloaded', () => {
    win.webContents.send('update-status', { step: 'ready' });
  });

  ipcMain.handle('check-for-updates', () => autoUpdater.checkForUpdates());
  ipcMain.handle('start-download', () => autoUpdater.downloadUpdate());
  ipcMain.handle('quit-and-install', () => autoUpdater.quitAndInstall());
}

module.exports = { setupAutoUpdater };
```

#### 自动更新全生命周期原理解析

- **1. 检查更新 (check-for-updates)**: 主进程请求 GitHub Releases 接口获取 latest.yml 配置文件, 比对本地与远端语义化版本号。
- **2. 静默下载 (start-download)**: 通过 download-progress 实时向渲染进程派发下载进度 (0%~100%), 前端可在 UI 呈现平滑进度条。
- **3. 完整性校验与就绪 (update-downloaded)**: 更新包下载完毕后自动比对 SHA512 哈希指纹, 防止篡改与网络损坏, 就绪后提示用户重启。
- **4. 退出并安装 (quit-and-install)**: 主进程退出当前应用, 静默调用 NSIS 安装程序完成原位覆盖替换, 并自动拉起全新版本。

## 阶段五：配置打包与一键发布

### 1. 在根目录 package.json 配置打包规则

```
JSON — package.json (根目录完整打包与更新分发配置)

{
  "name": "xc-omnibox",
  "version": "1.0.0",
  "main": "electron/main.js",
  "scripts": {
    "dev:electron": "concurrently \"npm --prefix frontend run dev\" \"wait-on http://localhost:3000 && electron .\"",
    "build:frontend": "npm --prefix frontend run build",
    "build:backend": "cd backend && pyinstaller --noconfirm --onedir --name omni-backend app/main.py",
    "dist": "npm run build:frontend && npm run build:backend && electron-builder"
  },
  "build": {
    "appId": "com.xc.omnibox",
    "productName": "XC 万象箱",
    "directories": {
      "output": "release"
    },
    "extraResources": [
      {
        "from": "backend/dist/omni-backend",
        "to": "backend"
      }
    ],
    "publish": [
      {
        "provider": "github",
        "owner": "LEESC88",
        "repo": "XC_OmniBox"
      }
    ],
    "win": {
      "target": ["nsis"],
      "icon": "resources/icon.ico"
    },
    "nsis": {
      "oneClick": true,
      "perMachine": false,
      "createDesktopShortcut": true,
      "createStartMenuShortcut": true,
      "shortcutName": "XC 万象箱"
    }
  }
}
```

#### package.json 关键配置字段解析

- **extraResources**: 将 PyInstaller 打包好的 Python 后端文件夹完整复制到安装目录的 resources/ 资源文件夹下，客户端直接调用，免除外部依赖。
- **publish (provider: github)**: 声明版本发布的远程仓库，electron-updater 会自动请求该 GitHub 仓库的 Releases 接口对比 latest.yml。
- **nsis (oneClick: true)**: 启用一键极速静默安装，无需向导式弹窗，自动创建桌面及开始菜单快捷方式，带来现代主流软件的用户体验。

## 以后你的两套工作流

### 工作流 A：日常写代码（99% 的时间）

```
● ● ● BASH — 终端命令：启动热更新开发环境
```

```
npm run dev:electron
```

- 一行命令同时拉起 Next.js 和桌面测试窗口。
- 在 VS Code 里改代码，桌面窗口直接热刷新，按 **Ctrl + S** 立即看到效果。

### 工作流 B：发布新版本（要给别人更新时）

- 1. 把 `package.json` 里的版本号改成 **1.0.1**。
- 2. 运行打包命令：

```
● ● ● BASH — 终端命令：打包发布生产安装包
```

```
npm run dist
```

- 3. 1~2 分钟后，`release/` 文件夹下会生成：
  - **XC 万象箱 Setup 1.0.1.exe**（安装包）
  - `latest.yml`（版本更新配置文件）
- 4. 把这两个文件上传到你 GitHub 仓库的 **Releases** 页面发布。

#### 自动更新升级效果

所有用户打开电脑上的老版本软件时，前端就会收到通知，点击“下载更新”，下载完毕点击“重启”，软件自动完成替换并升级到 **1.0.1**！

#### 方案结语

这就是完整的工业级桌面端方案。整个骨架非常清晰，我们可以一步一步来实施。